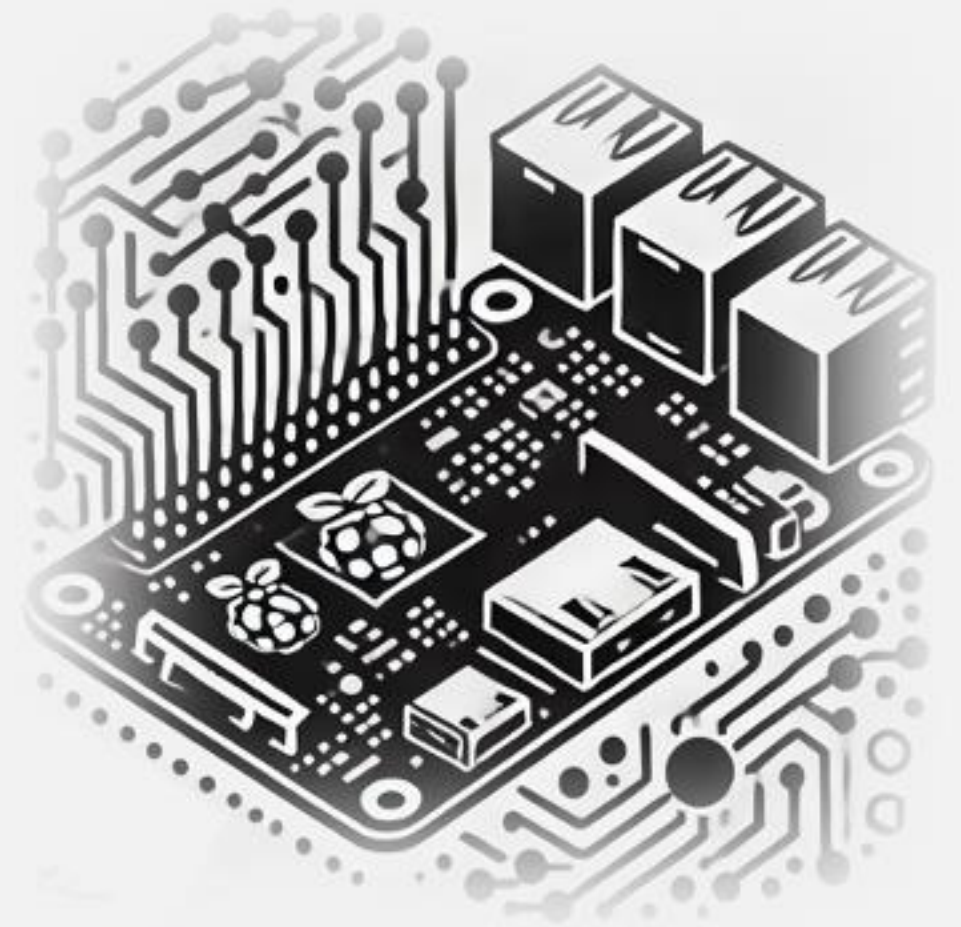


IESTI05 – Edge AI

Machine Learning System Engineering

5. Image Classification: Introduction to AI Edge LiteRT



Computer Vision Recognition Tasks

Image Classification (Multi-Class Classification)

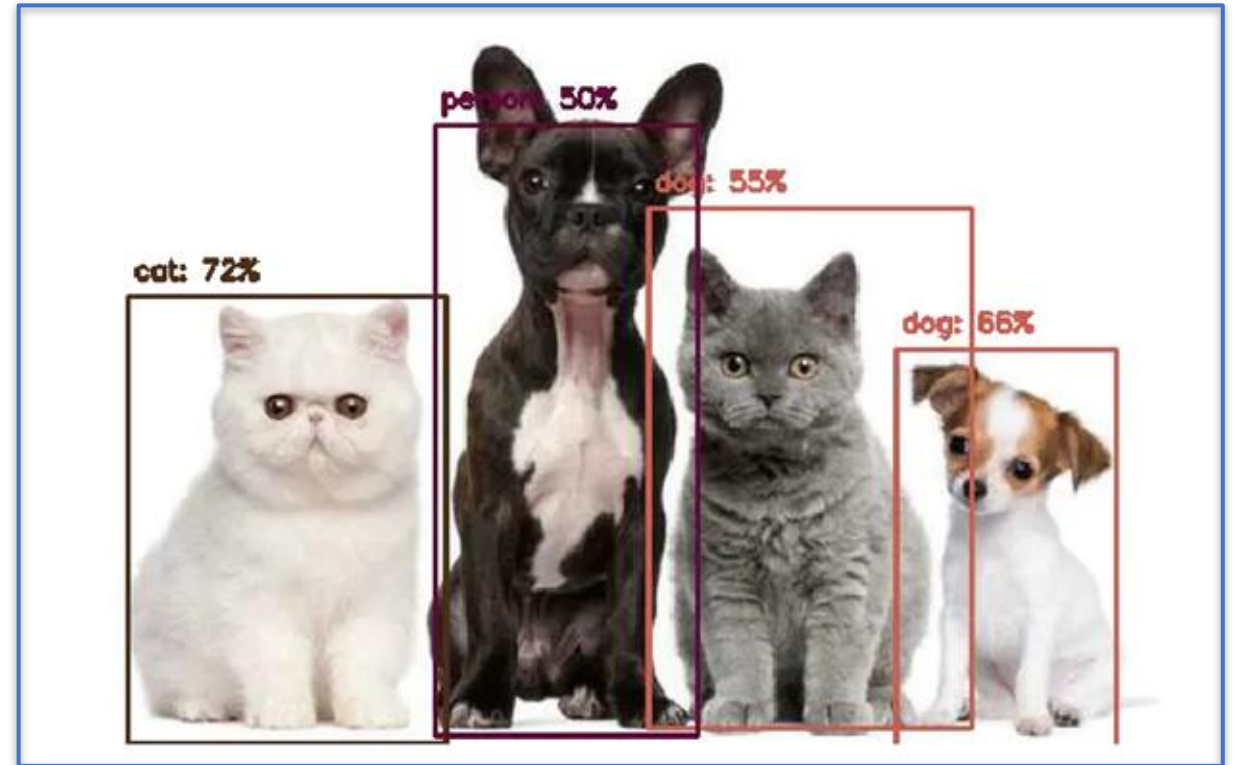


Cat: 70%



Dog: 80%

Object Detection Multi-Label Classification + Object Localization



Computer Vision Recognition Tasks

Instance Segmentation

Each pixel in an image IS CLASSIFIED into a predefined category.



Pose Estimation

Key points (or landmarks) on the object, such as joints on a human body are detected



Image Classification

What is Image Classification?

Definition

Image classification is a fundamental task in computer vision that involves **categorizing an image into one of several predefined classes**.

Key Characteristics

- Assigns a **single label to the entire image**
- Based on visual content analysis
- Uses machine learning algorithms
- Mimics human visual perception



Cat: 70%



Dog: 80%

Real-World Applications

Healthcare

Medical image analysis, X-ray and MRI diagnostics

Agriculture

Crop health monitoring, disease detection

Automotive

Autonomous vehicles, road sign recognition

Security

Surveillance systems, threat detection

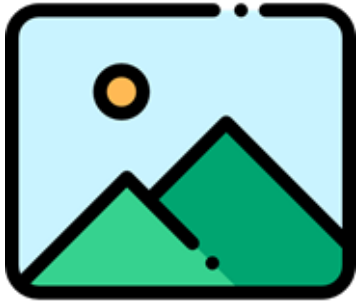
Retail

Visual search, inventory management

Environmental

Satellite imagery analysis, monitoring

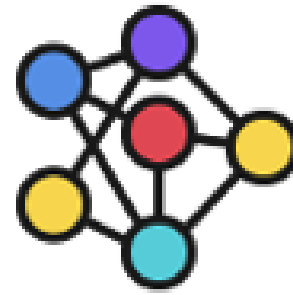
Image Classification: Inference Pipeline



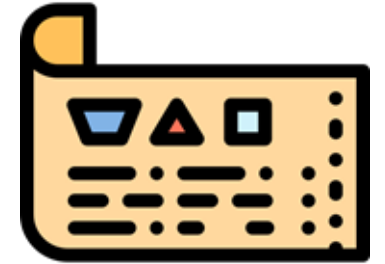
Input Image



Pre-Process

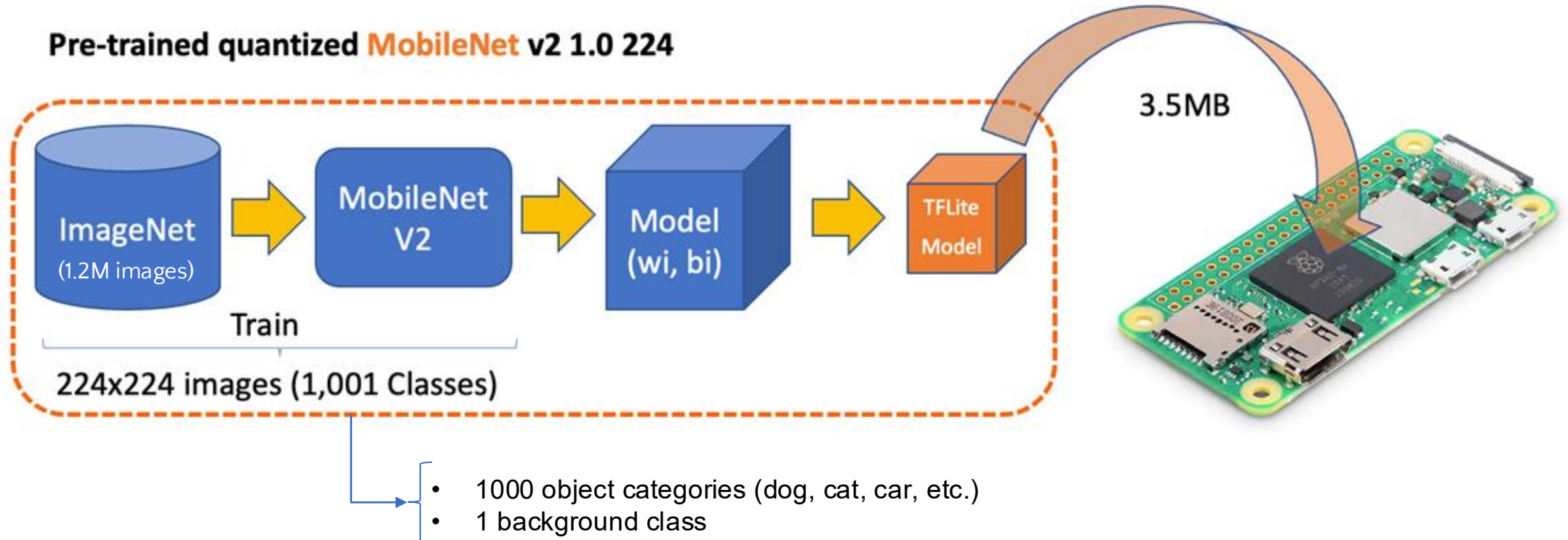


Classification
Model

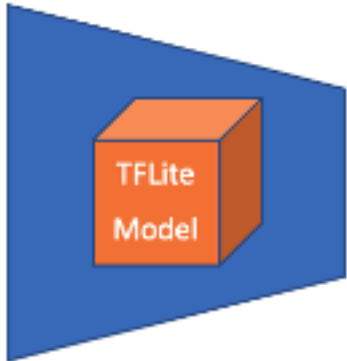


Predict Label

The MobileNet V2 model



TFLite Interpreter



input_details

```
[{'name': 'input',  
  'index': 171,  
  'shape': array([ 1, 224, 224,  3], dtype=int32),  
  'shape_signature': array([ 1, 224, 224,  3], dtype=int32),  
  'dtype': numpy.uint8,  
  'quantization': (0.0078125, 128),  
  'quantization_parameters': {'scales': array([0.0078125], dtype=float32),  
  'zero_points': array([128], dtype=int32),  
  'quantized_dimension': 0},  
  'sparsity_parameters': {}}]
```

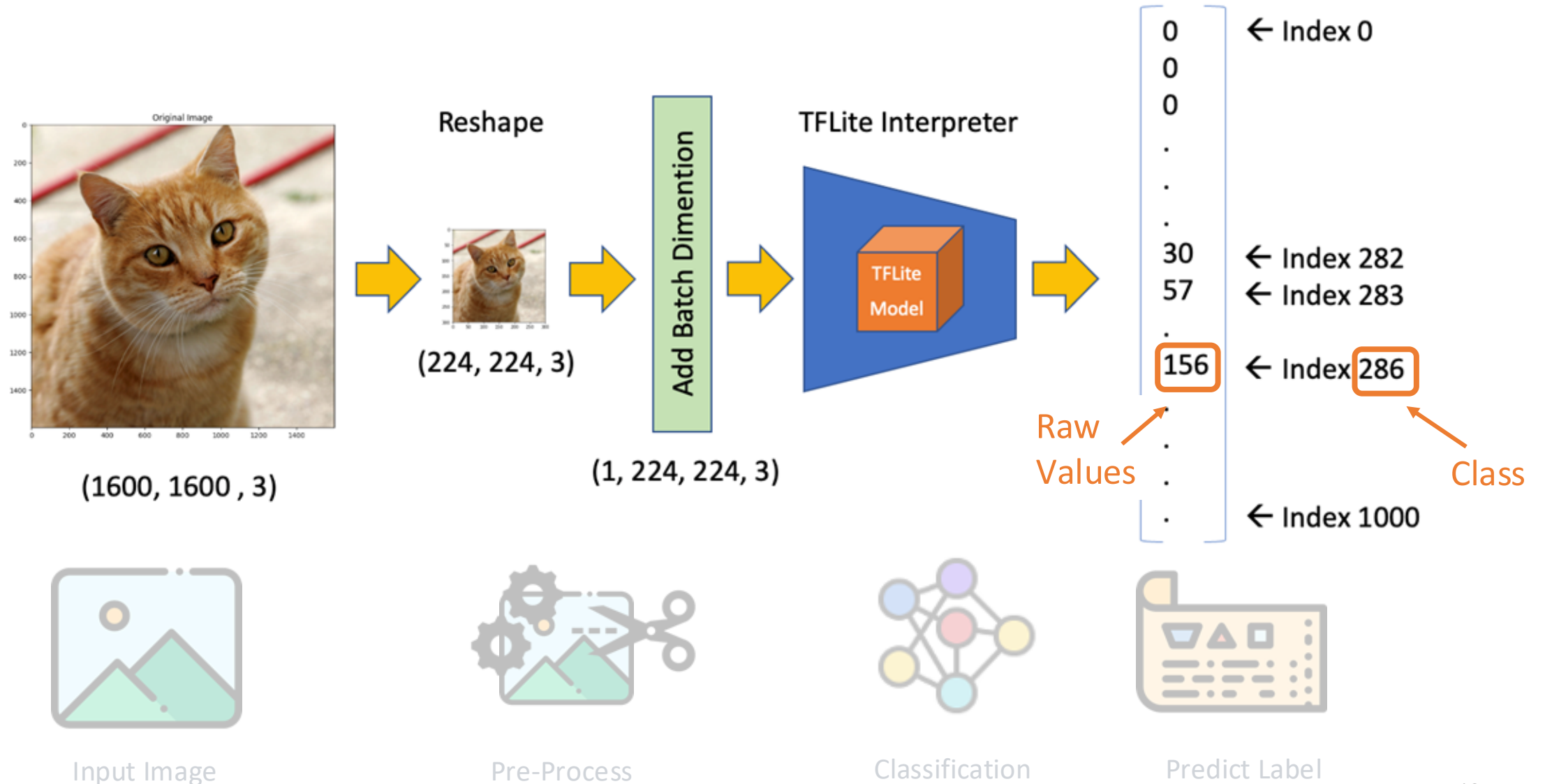
Input Image
Shape Signature
Pixel data type
Quantization Parameters

output_details

```
[{'name': 'output',  
  'index': 172,  
  'shape': array([ 1, 1001], dtype=int32),  
  'shape_signature': array([ 1, 1001], dtype=int32),  
  'dtype': numpy.uint8,  
  'quantization': (0.09889253973960876, 58),  
  'quantization_parameters': {'scales': array([0.09889254], dtype=float32),  
  'zero_points': array([58], dtype=int32),  
  'quantized_dimension': 0},  
  'sparsity_parameters': {}}]
```

Output model
Quant Dim 0: One set of parameters for the entire output tensor

Making **inferences** with MobileNet V2



LiteRT Runtime

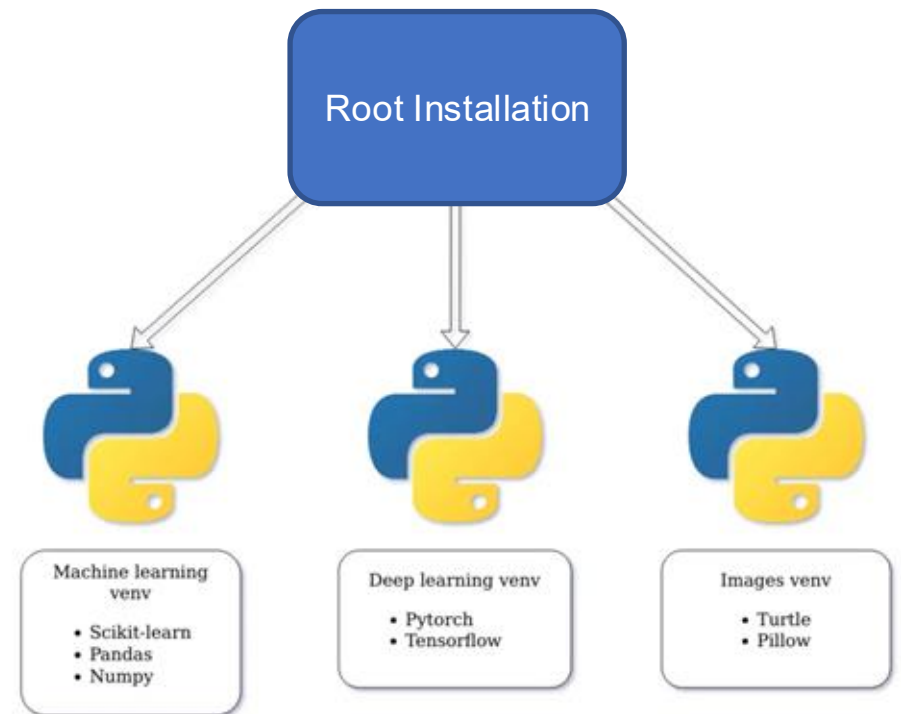
Setting up a Virtual Environment

Activate the environment:

```
source ~/tflite_env/bin/activate
```

To **exit** the virtual environment, use:

```
deactivate
```



LiteRT Setup

What is LiteRT?

Google's on-device framework for high-performance ML & GenAI deployment on edge platforms, via efficient conversion, runtime, and optimization.

We'll use the [LiteRT runtime](#), an advanced GPU/NPU acceleration which delivers superior ML & GenAI performance, making on-device ML inference easier than ever.

```
pip install ai-edge-litert
```

Successfully installed ai-edge-litert-2.2.0 backports.strenum-1.2.8 flatbuffers-25.12.19 ml_dtypes-0.6.0 protobuf-7.35.1

```
sudo reboot
```

Verify installation:

```
pip show ai-edge-litert
```

A terminal window screenshot showing the command 'pip show ai-edge-litert' and its output. The terminal title bar indicates the user is 'marcelo_rovai' on a 'raspi-zero2w-26' machine, connected via SSH. The output provides details about the 'ai-edge-litert' package, including its version (2.2.0), summary, home page, author, license, location, and required dependencies.

```
(tflite_env) mjrovai@raspi-zero2w-26:~$ pip show ai-edge-litert
Name: ai-edge-litert
Version: 2.2.0
Summary: LiteRT is for mobile and embedded devices.
Home-page: https://www.tensorflow.org/lite/
Author: Google AI Edge Authors
Author-email: packages@tensorflow.org
License: Apache 2.0
Location: /home/mjrovai/tflite_env/lib/python3.13/site-packages
Requires: backports.strenum, flatbuffers, ml_dtypes, numpy, protobuf, tqdm, typing-extensions
Required-by:
(tflite_env) mjrovai@raspi-zero2w-26:~$
```

Creating a **working directory** & get the **model**

```
Documents/  
├── TFLITE/  
│   └── IMG_CLASS/  
│       ├── models/  
│       │   ├── mobilenet_v2.tflite  
│       │   └── labels.txt  
│       └── images/  
│           └── test_image.jpeg
```

You only need the
.tflite and the
labels.txt files.

You can delete the
other files in the
models directory.

Go to the **models/** folder and get the Model and labels

```
wget
```

```
https://storage.googleapis.com/download.tensorflow.org/models/tflite\_11\_05\_08/mobilenet\_v2\_1.0\_224\_quant.tgz
```

```
tar xzf mobilenet_v2_1.0_224_quant.tgz
```

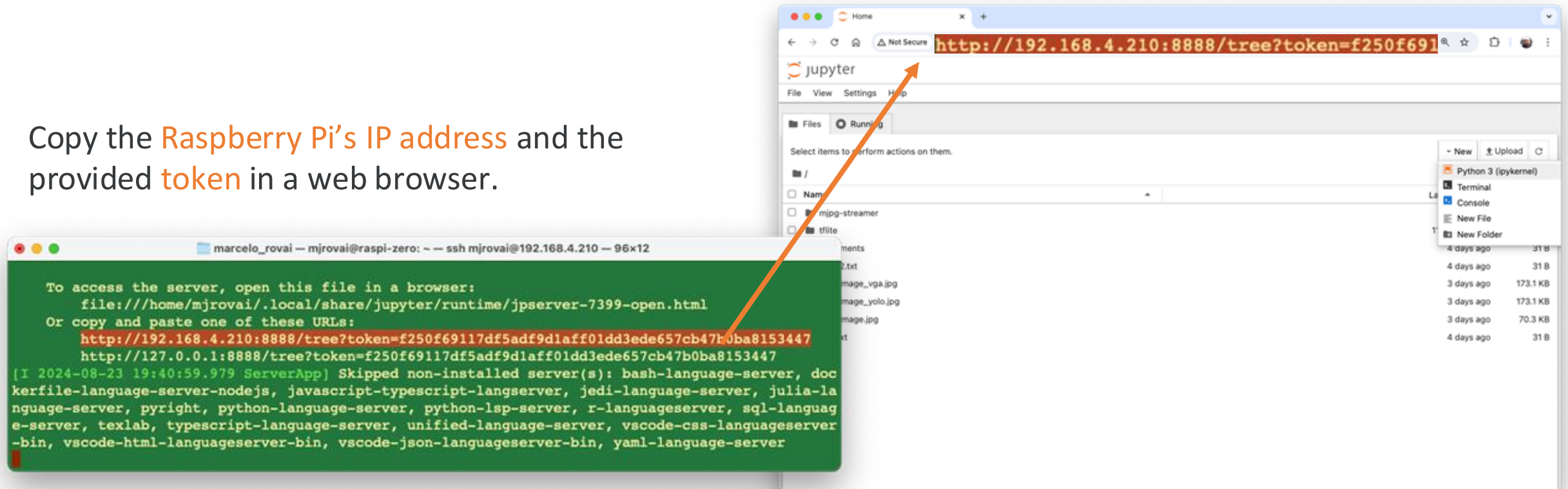
```
wget https://raw.githubusercontent.com/Mjrovai/EdgeML-with-Raspberry-Pi/refs/heads/main/IMG\_CLASS/models/labels.txt
```


Running up Jupyter Notebook



```
jupyter notebook --ip=[YOUR IP ADDRESS] --no-browser
```

Copy the **Raspberry Pi's IP address** and the provided **token** in a web browser.



Verifying the Setup



```
from ai_edge_litert.interpreter import Interpreter
import numpy as np
from PIL import Image

print("NumPy:", np.__version__)
print("Pillow:", Image.__version__)

# Try to create a LiteRT Interpreter
model_path = "./models/mobilenet_v2_1.0_224_quant.tflite"
interpreter = Interpreter(model_path=model_path)
interpreter.allocate_tensors()
print("LiteRT Interpreter created successfully!")
```

Note that on video, the runtime can show the old line:
import tflite_runtime.interpreter as tflite

Raspberry Pi Inference:

10_Image_Classification.ipynb



Making inferences: Static Images and Camera

[PREDICTION] [Prob]

tiger cat	: 37%
Egyptian cat	: 27%
tabby	: 16%
lynx	: 10%
carton	: 2%



[PREDICTION] [Prob]

coffee mug	: 99%
cup	: 0%
whiskey jug	: 0%
teapot	: 0%
water jug	: 0%



Questions?



Prof. Marcelo J. Rovai

rovai@unifei.edu.br



UNIFEI